

Arquitetura & Client-Side

Arquitetura Web & Programação Client-Side

Prof.^a Catarina Silva

Ano Letivo 2025/2026 – 2º Semestre

Objetivos da Aula

- Compreender a arquitetura base da World Wide Web (Modelo Cliente-Servidor).
- Desmistificar o protocolo HTTP e o papel dos Web Servers.
- Introduzir os fundamentos da programação do lado do cliente (Client-Side).
- Aprender a sintaxe base, variáveis, tipos de dados e controlo de fluxo em JavaScript.

Agenda

1 Arquitetura Web e HTTP

2 Programação Client-Side

O que é a Web?

- A Web é um serviço que corre sobre a Internet.
- Baseia-se na partilha de documentos interligados (hipertexto).
- Funciona através de um modelo fundamental: **Cliente - Servidor**.

O Modelo Cliente-Servidor

- **Cliente:** Aquele que pede a informação (ex: o nosso Web Browser - Chrome, Firefox).
- **Servidor:** Aquele que aloja a informação e responde aos pedidos (ex: num datacenter).
- *Prática:* Pense num restaurante. O Cliente faz o pedido (browser), o Empregado anota o pedido (rede/protocolo), a Cozinha prepara a refeição (servidor).

O que é o Protocolo HTTP?

- **HTTP** = *Hypertext Transfer Protocol*.
- É a linguagem que o Cliente e o Servidor usam para comunicar.
- É um protocolo *stateless* (sem estado): cada pedido é independente, o servidor não se lembra de pedidos passados por defeito.

O Ciclo Pedido/Resposta (Request/Response)

- 1 O Cliente envia um **HTTP Request** (Pedido).
- 2 O Servidor processa o pedido.
- 3 O Servidor devolve um **HTTP Response** (Resposta).
- 4 O Cliente (Browser) renderiza o resultado no ecrã.

Anatomia de um HTTP Request

- **Método (Method):** A ação pretendida (ex: GET).
- **URL:** O endereço do recurso pretendido.
- **Headers (Cabeçalhos):** Metadados adicionais (ex: tipo de browser).
- **Body (Corpo):** Dados enviados (ex: formulário). *Nota: Ausente em GET simples.*

Os Principais Métodos HTTP

- **GET:** Pedir dados ao servidor (Ler).
- **POST:** Enviar dados para o servidor criar algo novo (Criar).
- **PUT / PATCH:** Atualizar dados existentes no servidor (Atualizar).
- **DELETE:** Apagar dados no servidor (Apagar).

Anatomia de um HTTP Response

- **Status Code (Código de Estado):** Indica se o pedido correu bem ou falhou.
- **Headers:** Metadados da resposta (ex: tipo de conteúdo - HTML, JSON).
- **Body:** O conteúdo propriamente dito (ex: o código HTML da página web).

Códigos de Estado HTTP (Status Codes)

- **1xx (Informativo):** Pedido recebido, a processar.
- **2xx (Sucesso):** O pedido foi aceite e processado (ex: **200 OK**).
- **3xx (Redirecionamento):** O recurso mudou de sítio.
- **4xx (Erro do Cliente):** O cliente fez algo errado (ex: **404 Not Found**).
- **5xx (Erro do Servidor):** O servidor falhou a processar um pedido (ex: **500 Internal**).

O que é um Web Server?

- **Hardware:** O computador físico onde os ficheiros do website estão guardados.
- **Software:** O programa que percebe os URLs e o protocolo HTTP, gerindo como aceder aos ficheiros alojados (ex: Apache, Nginx, IIS).

Conteúdo Estático vs. Dinâmico

- **Estático:** O servidor envia o ficheiro exato que está no disco. Todos veem o mesmo.
- **Dinâmico:** O servidor corre código (PHP, Node.js) e consulta bases de dados para "montar" o HTML em tempo real antes de o enviar.

Resumo da Arquitetura

Browser → HTTP GET → Web Server



Processamento



Browser renderiza ← HTTP 200 OK + HTML ← Web Server

Perguntas sobre Arquitetura?

Dúvidas?

O que é Programação Client-Side?

- É o código que é executado no computador do utilizador final (no Browser).
- **Objetivo:** Adicionar interatividade, validar formulários, alterar a interface.
- **A Linguagem Standard:** JavaScript (JS).

O Trio da Web Front-End

- **HTML:** Estrutura
- **CSS:** Apresentação
- **JavaScript:** Comportamento.

Como incluir JavaScript no HTML?

- **Inline:** Diretamente nos elementos. *Não recomendado.*
- **Interno:** Dentro da tag `<script>` no ficheiro HTML.
- **Externo:** Ficheiro separado `.js` incluído no HTML. *Melhor prática.*

```
<script src="script.js"></script>
```

Sintaxe Básica e Regras (JavaScript)

- **Case Sensitive:** Nome é diferente de nome.
- **Ponto e vírgula (;):** Indica o fim de uma instrução.
- **Comentários:**
 - `//` Comentário de uma linha
 - `/*` Comentário de múltiplas linhas `*/`

Ferramentas do Desenvolvedor

- Todos os browsers modernos têm a Consola.
- Atalho: F12 ou Ctrl+Shift+I / Cmd+Option+I.
- **Prática:** Como imprimir mensagens na consola para testes?

```
console.log("Ola, Turma!");
```

O que são Variáveis?

- Uma variável é como um espaço na memória do computador onde guardamos dados.
- Este "espaço" tem nomes (identificadores) e conteúdos (valores).

Declarar Variáveis (let, const, var)

- **let**: Permite declarar uma variável cujo valor pode mudar no futuro.
- **const**: Declara uma constante. O valor não pode ser reatribuído.
- **var**: A forma antiga. *Evitar usar em código moderno.*

```
let idade = 20;  
const pi = 3.14;
```

Regras para Nomear Variáveis

- Podem conter letras, números, _ (underscore) ou \$ (dólar).
- **Não** podem começar por números.
- Convenciona-se usar *camel/Case*: nomeDoAluno, anoNascimento.

Tipos de Dados (Data Types)

- Os computadores precisam de saber que tipo de informação estão a guardar.
- Dois grandes grupos em JS: **Primitivos** e **Objetos**.
- Vamos focar-nos nos Primitivos.

Tipos Primitivos: String (Texto)

- Representa texto. Envolvido em aspas duplas, simples ou backticks.

```
let nome = "Maria";  
let saudacao = 'Ola ' + nome;
```

Tipos Primitivos: Number (Números)

- Representa números, sejam inteiros ou decimais (floats).
- Em JS só existe o tipo Number.

```
let idade = 25;  
let preco = 19.99;
```

Tipos Primitivos: Boolean

- Representa uma entidade lógica que só pode ter dois valores: `true` ou `false`.
- Essencial para tomar decisões no código (condicionais).

```
let estaAprovado = true;
```

Undefined e Null

- `undefined`: Uma variável foi declarada, mas ainda não lhe foi atribuído nenhum valor.
- `null`: A ausência intencional de um valor. Esvaziámos a caixa de propósito.

Operadores: O que são?

- Símbolos que dizem ao interpretador para realizar uma operação matemática, relacional ou lógica específica.

Operadores Aritméticos

- Adição (+), Subtração (-), Multiplicação (*), Divisão (/), Módulo (%)

```
let total = 10 + 5 * 2; // 0 resultado e 20
```

Operadores de Atribuição

- O operador de atribuição básico é o `=`. (Não confunda com igualdade!)
- Atribuições compostas: `+=`, `-=`.
- Incremento e Decremento: `x++`, `x--`.

Operadores de Comparação

- Comparam dois valores e devolvem um Boolean.
- Maior / Menor: >, <, >=, <=
- **Igualdade:** == (compara valor) vs. === (compara valor e tipo). **Usar sempre ===.**

Operadores Lógicos

- **AND (&&):** Verdadeiro apenas se AMBAS as condições forem verdadeiras.
- **OR (||):** Verdadeiro se PELO MENOS UMA condição for verdadeira.
- **NOT (!):** Inverte o valor.

Controlo de Fluxos

- O código lê-se de cima para baixo, linha a linha.
- O controlo de fluxos altera isto permitindo:
 - Saltar blocos de código (Condicionais)
 - Repetir blocos de código (Ciclos/Loops)

Condicionais (if / else if / else)

- Permite executar código apenas se uma condição for verdadeira.

```
let nota = 12;  
if (nota >= 10) {  
    console.log("Aprovado");  
} else {  
    console.log("Reprovado");  
}
```

Condicionais (Switch)

- Alternativa a muitos `if...else` seguidos para avaliar uma única variável com valores fixos.

```
switch(diaDaSemana) {  
    case 1: console.log("Segunda"); break;  
    // ...  
}
```

Ciclos / Loops (Porquê usá-los?)

- Princípio DRY: *Don't Repeat Yourself*.
- Serve para executar o mesmo bloco de código várias vezes, enquanto uma condição for verdadeira.

Ciclo: FOR

- Utilizado quando sabemos exatamente quantas vezes queremos que o ciclo repita.

```
for (let i = 1; i <= 5; i++) {  
    console.log("O valor e: " + i);  
}
```

Ciclos: WHILE

- Utilizado quando queremos repetir *enquanto* uma condição for verdadeira, sem saber o número de repetições.
- **Perigo:** *Infinite Loops!* Garantir que a condição se torna falsa.

Perguntas sobre JS?

Dúvidas?

EXTRA sobre JS

- O tipo da variável é inferido em tempo de execução e pode mudar.
- **Coerção de Tipos:** O JS tenta converter tipos automaticamente, o que pode causar resultados inesperados.

Exemplo Clássico (Cuidado!):

- `1 + "1"` resulta em `"11"` (o número 1 é convertido para String).
- *Por isto usamos sempre `===` para comparar (compara valor e tipo)!*

Introdução às Funções

- Uma **função** é um bloco de código desenhado para executar uma tarefa específica.
- Permite encapsular lógica e reutilizar código (*Princípio DRY*).
- Executa apenas quando é invocada/chamada.

```
// Declaracao da funcao
function saudar(nome) {
    return "Ola, " + nome + "!";
}

// Invocacao
let mensagem = saudar("Turma");
console.log(mensagem);
```

Como o JS interage com a página? (O DOM)

- Já sabemos HTML, CSS e JS. Mas como se ligam?
- **DOM (Document Object Model):** Quando o browser carrega um HTML, ele cria uma representação estruturada em forma de árvore (*Tree*).
- O JavaScript usa o DOM para aceder, alterar, adicionar ou remover elementos HTML e estilos CSS em tempo real.
- *Este será o foco principal das nossas próximas aulas práticas!*

A Ponte: Onde entra o JavaScript?

- Até agora, o nosso HTML era **estático**: uma vez carregado pelo browser, não mudava.
- O JavaScript atua como o **manipulador** da página.
- Ele consegue ler o HTML, alterar textos, mudar cores no CSS, apagar elementos ou criar coisas novas *depois* da página já estar carregada.
- Mas como é que o JS "vê" o HTML? Através do **DOM**.

O que é o DOM (Document Object Model)?

- Quando o browser lê o ficheiro HTML, ele converte as tags numa estrutura de dados na memória chamada **Árvore DOM** (*DOM Tree*).
- O documento inteiro é o objeto document.
- Cada tag HTML (<html>, <body>, <h1>, <p>) transforma-se num **Nó** (*Node*) ou **Objeto** nessa árvore.
- O JavaScript consegue interagir diretamente com esta árvore!

Passo 1: Encontrar Elementos HTML (Seletores)

- Para o JS alterar uma tag, primeiro tem de a encontrar no DOM.
- Usamos o objeto global document e os seus métodos de pesquisa.

```
// Selecionar pelo atributo ID (o mais comum)
let meuTitulo = document.getElementById("tituloPrincipal");

// Selecionar de forma semelhante ao CSS moderno
let primeiroParagrafo = document.querySelector(".texto-destaque");
```

Passo 2: Manipular Elementos (Conteúdo e Estilo)

- Depois de guardar a referência do HTML numa variável JS, podemos alterar as suas propriedades.

```
let caixa = document.getElementById("caixaMensagem");  
  
// Alterar o texto interno do HTML  
caixa.innerHTML = "Bem-vindo a aula de Tecnologias Web!";  
  
// Alterar o estilo CSS diretamente  
caixa.style.color = "blue";  
caixa.style.backgroundColor = "lightgray";
```

Passo 3: Escutar o Utilizador (Eventos)

- A verdadeira interatividade acontece quando o JS reage a ações do utilizador (**Eventos**), como cliques, passar o rato ou escrever no teclado.

```
let botao = document.getElementById("btnAcao");

// Adicionar um "escutador de eventos" (Event Listener)
botao.addEventListener("click", function() {
    alert("O botao foi clicado!");
    // Aqui dentro colocamos a logica que altera o DOM
});
```


A Grande Questão: Porquê usar JS?

Pergunta comum: *"Se eu quero que o texto seja vermelho ou que o botão diga 'Clicado', porque não escrevo logo isso no HTML e no CSS?"*

A Resposta: Tempo e Interatividade!

- **HTML/CSS:** Definem o estado **inicial** da página (o Ponto A).
- **JavaScript:** Define o estado **futuro** da página (o Ponto B), reagindo ao que acontece *depois* da página carregar.

O HTML constrói a casa. O JavaScript acende a luz apenas quando o utilizador entra na divisão.

O que aconteceria se não usássemos JS?

- **Qualquer pequena alteração** (abrir um menu, dar um "Like", validar se uma password está forte) obrigaria o Browser a pedir uma página HTML inteiramente nova ao Servidor.

Vantagens do JavaScript (Client-Side):

- **Rapidez:** Altera apenas pequenas partes do ecrã instantaneamente.
- **Zero Recarregamentos:** O utilizador não vê a página a "piscar" ou a recarregar.
- **Lógica Complexa:** O HTML não sabe fazer contas (1+1), o JS sim!